

Course 3 Notes: Unsupervised Learning, Recommenders, Reinforcement Learning

Andrew Ng — Jesse Hernandez

DeepLearning.AI/Stanford Online — 2026

My notes for the third course in a three part machine learning specialization course on coursera consisting of three learning modules.

- Course objectives:

- Unsupervised learning
 - Clustering
 - Anomaly Detection
- Recommender Systems
- Reinforcement Learning (new and emerging)

1. Week 1: Unsupervised learning

1.1 Overview

- Key objectives

- Introduction to unsupervised learning
- Clustering (k-mean algorithm)
- Anomaly detection algorithm

- Applications

- Optimization and chose for k-means for clustering
- Gaussian distributions
- Developing and evaluating anomaly detection systems
- comparing anomaly detection vs supervised learning

Definition

Unsupervised learning is a learning algorithm that takes as input unlabeled data $\{x^{(1)}, x^{(2)}, x^{(3)}, \dots, x^{(m)}\}$ and finds structures in the data

1.2 Clustering

Definition

Clustering is an unsupervised learning algorithm that finds data points that are related or similar to each other

– Some applications:

- grouping similar news stories in searches
- dna analysis for grouping similar genetics
- in astronomical data clustering is used to group celestial bodies in data

K-means clustering intuition given a sample data set to find two clusters (number of cluster choice to be discussed later):

- The algorithm chooses two guess points for the centroids of the clusters in the sample space to initialize on and assigns data points to the closest(distance) centroid point
- A new centroid is calculated from the mean of the previously assigned data and the data points are reassigned based on proximity to new centroid
- repeat the above steps until there are no more changes to the centroid and the data is clustered

Formalism

K-means clustering algorithm succinctly mathematically explained in steps:

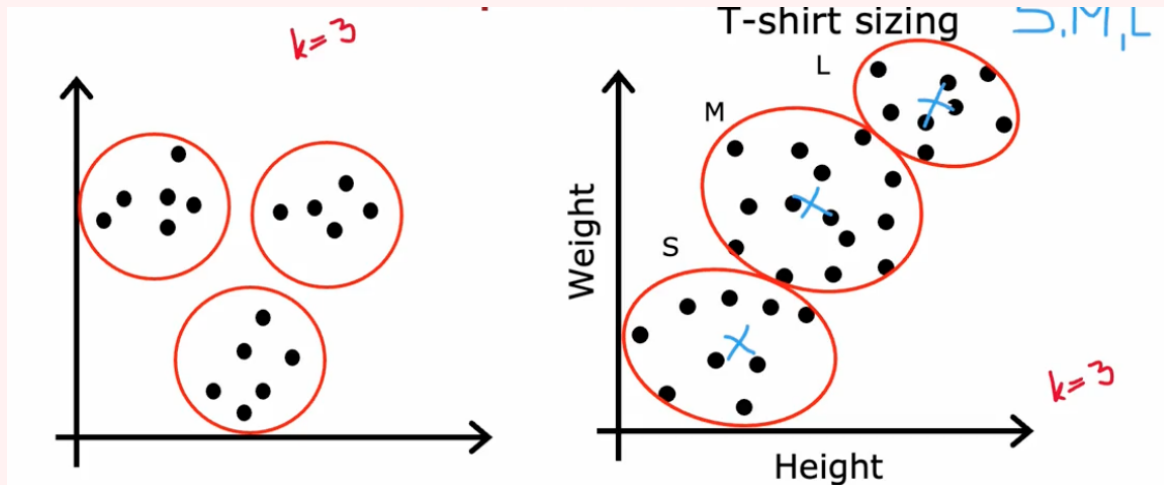
- Randomly initialize K cluster centroids $(\mu_1, \mu_2, \mu_3, \dots, \mu_K)$ where μ_i have the dimensions of the training features

```
Repeat{
  1.) assign points to cluster centroids
  for i = 1 to m
    #represents which cluster to assign to
    c_i := index (1 to K) of cluster centroid closest to point x_i
  2.) move cluster centroids
  for k = 1 to K
    mu_k := average(mean) of points assigned to cluster k
}
```

where the centroid is calculated by minimizing the norm of the squared distance to the points $\min_k \|x^{(i)} - \mu_k\|^2$ and the average is $\mu_i = \frac{1}{N} \sum x^{(i)}$

Note

K-means clustering for data that is not well separated can still do well. For example when identifying sizing of t-shirts for clients(right image) we see below the trend is not well separated but the k-means finds good regions to fit sizing for based on weight.

**1.2.1 K-means****Formalism**

The clustering algorithm is **optimizing** for the distance of the nearest points and has **cost functions**:

$$J(c^{(1)}, \dots, c^{(m)}, \mu_1, \dots, \mu_K) = \frac{1}{m} \sum_{i=1}^m \|x^{(i)} - \mu_{c^{(i)}}\|^2 \quad (1)$$

where $\mu_{c^{(i)}}$ is the cluster centroid to which example $x^{(i)}$ has been assigned. Also sometimes called the **distortion function**.

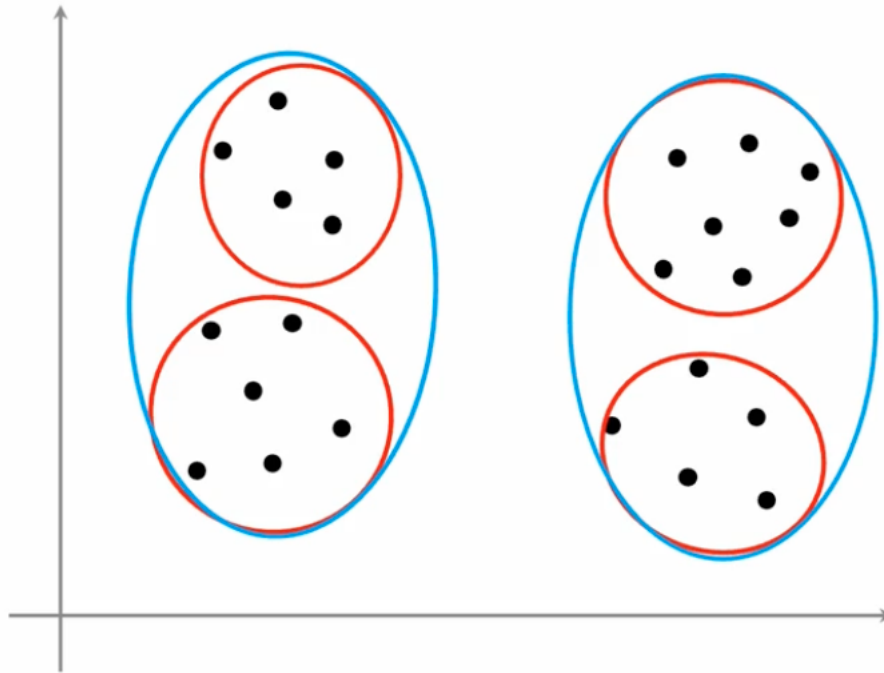
Formalism

Initialization of k-means can be sensitive to the clustering pattern discovered by the algorithm (could find local minima instead of global ones). To make sure we choose the best configuration we:

- Run the clustering algorithm N times with randomly initialized k-means and compute the cost function (store)
 - As expected the clustering pattern with the smallest cost function is chosen as the best one
- Recommendation for efficiency would be to run around 50-100 times
 - This method of repeating the algorithm does much better than running it once

1.2.2 Choosing K

When choosing the number of clusters to look for in data it can be truly ambiguous or hard to disentangle. Provided the picture below we see that there are many choices or possibilities that 'look' good to the eye.

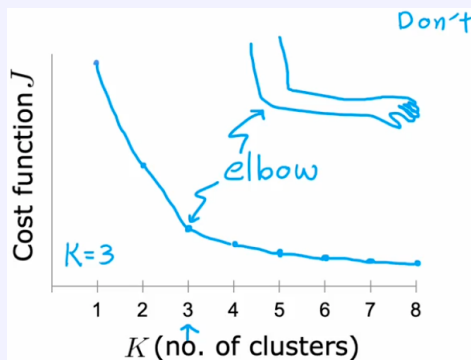


How do we then choose the best choice??

Definition

- Some method used in the literature for k-means

The **elbow method** is a method used to choose K by running the algorithm over many K values and plotting it against the cost function J



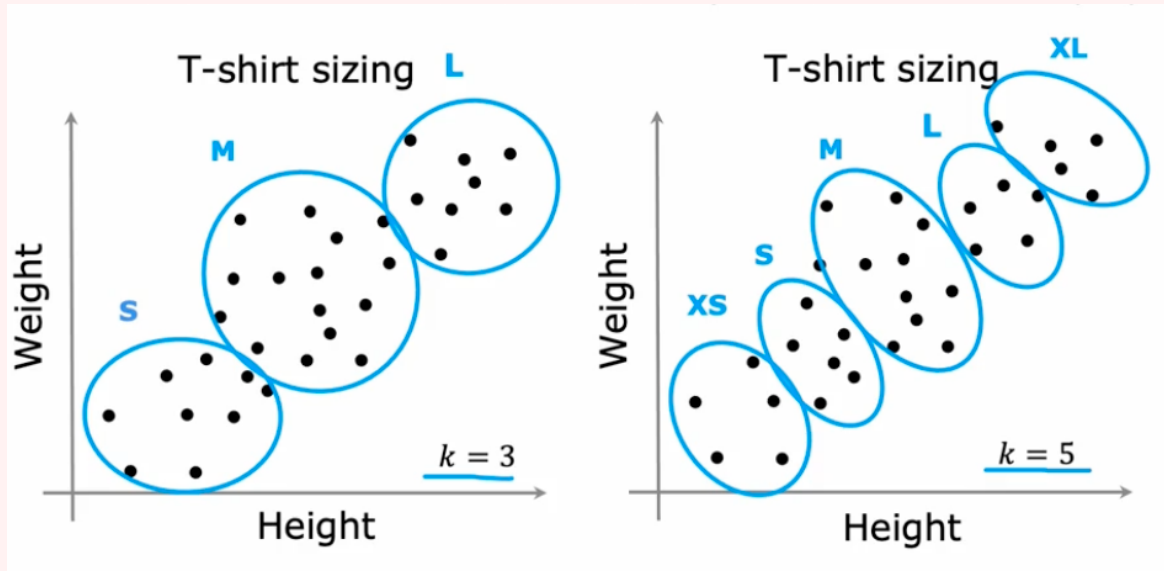
- The kink (rapid transition) in the plot gives us an indication of the right choice for K clusters
- Caveat being that sometimes this is just a smooth decreasing function with no kink to choose and so choosing the lowest cost is not good
- The choice of K is often ambiguous

Note

Recommended Choice of clusters K :

- Often you want to use the clusters for some downstream purpose
- Therefore, evaluate K-means based on performance for that later purpose

An example of this use case is provided below, where the choice depends on the trade off between having more sizes for better fit with increases cost or less sizes for simplicity and lower cost.



1.3 Anomaly Detection

Definition

Anomaly Detection is an algorithm that takes unlabeled 'normal' events and find/detects anomalies/outliers

Density estimation is the technique used in anomaly detection where for a given data set a model gives the probability of an event being in the dataset where high density of events = high probability, broken up into probability regions

Example

- Some examples of anomaly detection use cases:

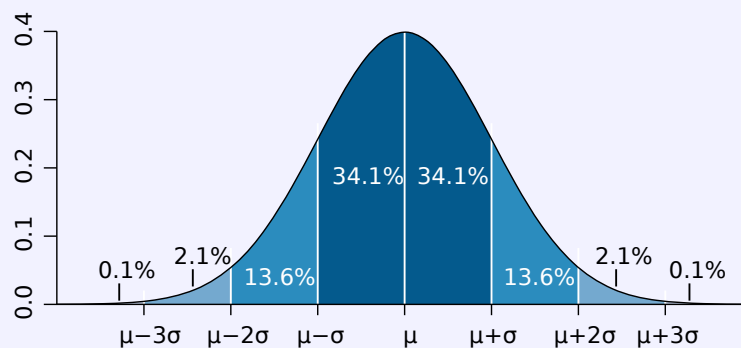
- Manufacturing (Quality Control): Features are product specs (e.g. airplane engine, circuit board)
- Fraud detection: Features are user's activity (e.g. how often they log in? typing speed?)
- Monitoring computers at data centers: Features of the machine (e.g. machine failure, possible hacks)

1.3.1 Gaussian (Normal) Distribution

Definition

A **Gaussian or Normal Distribution** is a continuous bell-shaped probability curve with its behavior defined by its mean μ and variance σ^2 (width is standard deviation σ), where most events are populated about the mean.

$$p(x) = \frac{1}{\sqrt{(2\pi)\sigma}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right) \quad (2)$$



Formalism

Parameter estimation for anomaly detection using the Gaussian distribution using the data $\{x^{(1)}, x^{(2)}, x^{(3)}, \dots, x^{(m)}\}$ by estimating the mean and variance in this way:

$$\mu = \frac{1}{m} \sum_{i=1}^m x^{(i)} \quad (3)$$

$$\sigma^2 = \frac{1}{m} \sum_{i=1}^m (x^{(i)} - \mu)^2 \quad (4)$$

also called the **Maximum likelihood estimation** for μ, σ .

1.3.2 Algorithm

Given

Training Set: $\{\vec{x}^{(1)}, \vec{x}^{(2)}, \vec{x}^{(3)}, \dots, \vec{x}^{(m)}\}$

(Each example $\vec{x}^{(i)}$ has n features)

Definition

Density estimation is constructing the probability density of the example assuming feature independence:

$$\begin{aligned} p(\vec{x}) &= p(x_1; \mu_1, \sigma_1^2) * p(x_2; \mu_2, \sigma_2^2) * \dots * p(x_n; \mu_n, \sigma_n^2) \\ &= \prod_{j=1}^n p(x_j; \mu_j, \sigma_j^2) \end{aligned} \quad (5)$$

Formalism

Anomaly Detection algorithm:

- Choose n features x_i that may be indicative of anomaly detection
- Fit parameters $\mu_1, \dots, \mu_n, \sigma_1^2, \dots, \sigma_n^2$ using equations Eqs. (3) and (4) including the j - th subscript
- Given new example x_{new} , compute $p(x)$ using Eq. (5) and ??, where

$$p(x) = \prod_{j=1}^n p(x_j; \mu_j, \sigma_j^2) = \prod_{j=1}^n \frac{1}{\sqrt{(2\pi)\sigma_j^2}} \exp\left(-\frac{(x_j - \mu_j)^2}{2\sigma_j^2}\right) \quad (6)$$

- Flag anomalous event based on a small threshold $p(x) < \epsilon$

1.3.3 Development and evaluation

Definition

Importance of **real number evaluations**.

Assuming that we have some labeled data, anomalous ($y = 1$) and non-anomalous ($y = 0$), examples we can construct training/cross validation/test sets of data in this way

- Training set: $\{\vec{x}^{(1)}, \vec{x}^{(2)}, \vec{x}^{(3)}, \dots, \vec{x}^{(m)}\}$ (assumed to have all not anomalous examples)

Note In practice a very small contamination does not affect the algorithm too much, e.g., anomalous events yet determined

- Cross validation set: $(x_{cv}^{(1)}, y_{cv}^{(1)}), \dots, (x_{cv}^{(m_{cv})}, y_{cv}^{(m_{cv})})$
- Test set: $(x_{test}^{(1)}, y_{test}^{(1)}), \dots, (x_{test}^{(m_{test})}, y_{test}^{(m_{test})})$

The cross validation and test sets contain some of the anomalous examples.

Example: Use the cross validation set to tune threshold for anomalous events ϵ and training data x_j then use the test set for evaluation.

Typical Alternative: Omit the test set and only have training/cross validation set (useful for limited data but without test set hard to evaluate on future data and can overfit)

Formalism

Algorithm Evaluation on the cross validation/test examples are labeled:

$$y = \begin{cases} 1 & \text{if } p(x) < \epsilon \text{ (anomaly)} \\ 0 & \text{if } p(x) > \epsilon \text{ (normal)} \end{cases} \quad (7)$$

We use the cross validation set to see how often its finding anomalies and choose parameter, ϵ

Possible evaluation metrics (optional skewed datasets [course 2 week3]):

- True positive, false positive, false negative, true negative
- Precision/Recall
- F₁-score

1.3.4 Anomaly detection vs Supervised learning

The advantages for the optimal use-case for either learning algorithm is summarized in the table below.

Anomaly detection	Supervised learning
Very small number of anomalous(positive) examples, large number of non-anomalous (negative) examples • For many 'types' of anomalies hard for algorithms to do well on training sets with only negative examples • Anomaly detection trains only on negative samples therefore can pick up on these possible future anomalies	Large number of positive and negative examples • We assume there are enough positive examples for the algorithm to decide well what positive events are • We assume that future positive examples are similar to the training set
Example use cases: • Fraud detection (many different types of future fraud) • Manufacturing to find new unseen defects • Monitoring machines in data centers (hackers finding new ways to hack into system)	Example use cases: • Email spam classification (similar types of spam emails to detect) • Manufacturing to find known, seen defects • Weather prediction (types of weather are known) • Disease classification

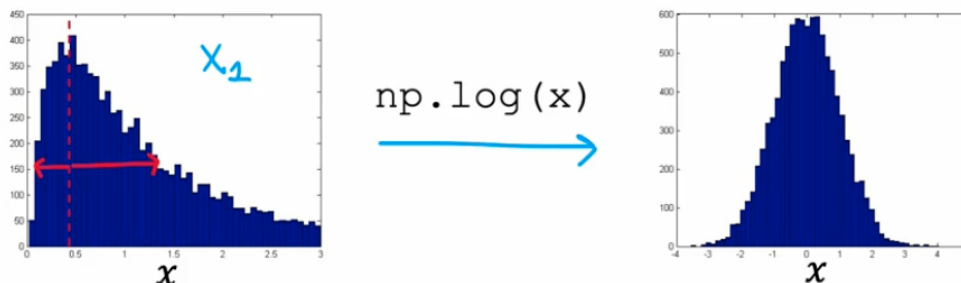
1.3.5 Choosing features

Turns out that the feature set for anomaly detection is very important.

How do we choose the best features?

Features should be gaussian

- We can handle **non-gaussian features** with transformations such as $x_1 \rightarrow \log(x_1)$, $x_2 \rightarrow \log(x_2 + C)$, $x_3 \rightarrow \sqrt{x_3}$.



Error analysis for anomaly detection, most common problems:

Problem: $p(x)$ is comparable for both anomalous and normal examples (fails to find the anomaly)

Solution: Directly look at failed examples to see if there are new features that may be used to distinguish the anomaly

1.4 Implementation

Graded Programming Assignments

- K-Means Clustering
 - Gain intuition of k-means on a sample dataset
 - Use k-means algorithm for image compression by reducing the number of colors in an image
- Anomaly Detection
 - Implement the anomaly detection algorithm and apply it to detect failing servers on a network

2. Week 2: Recommender Systems

- Key objectives
 - Introduce recommender systems
- Applications
 - Seen all over for many service like netflix etc

Definition

Recommender systems are learning algorithms that give recommendations based on user information, e.g., movie recommendation based on user ratings

Example

For reference we will use the following example data to learn about recommender systems where we have users($n_u = 4$) and movies($n_m = 5$) and some movie features (x_i):

What if we have features of the movies?

Movie	Alice(1)	Bob(2)	Carol(3)	Dave(4)	x_1 (romance)	x_2 (action)
Love at last	5	5	0	0	0.9	0
Romance forever	5	?	?	0	1.0	0.01
→ Cute puppies of love	?	4	0	?	0.99	0
Nonstop car chases	0	0	5	4	0.1	1.0
Swords vs. karate	0	0	5	?	0	0.9

$n_u = 4$
 $n_m = 5$
 $n = 2$

$x^{(1)} = \begin{bmatrix} 0.9 \\ 0 \end{bmatrix}$
 $x^{(3)} = \begin{bmatrix} 0.99 \\ 0 \end{bmatrix}$

2.1 Collaborative Filtering

2.1.1 Per-item features

How can we build a recommender system if we also had features for each item

- For movie recommender we can include features of the movie, e.g., romance, action, (table above)

Formalism

To build a recommendation we can predict the movie ratings of unseen movies for each user and recommend the highest rated prediction.

- Predict user j rating of movie i using a **linear regression** model with the user ratings as weights($w^{(j)}$) and the movie features:

$$w^{(j)} \cdot x^{(i)} + b^{(j)}$$

The notation to construct the cost function:

- $r(i, j) = 1(0)$ if user j has rated movie i
- $y^{(i,j)}$ is rating given by user j for movie i
- $w^{(j)}, b^{(j)}$ parameters for movie j
- $x^{(i)}$ are features of movie i
- $m^{(j)}$ number of movies rated by user j

Then the **recommender cost function**(squared errors cost function) to find the parameters with the regularization term (discussed in course 1, reduce overfitting) can be written as

$$J(w^{(j)}, b^{(j)}) = \frac{1}{2m^{(j)}} \sum_{i:r(i,j)=1} \left(w^{(j)} \cdot x^{(i)} + b^{(j)} - y^{(i,j)} \right)^2 + \frac{\lambda}{2m^{(j)}} \sum_{k=1}^n \left(w_k^{(j)} \right)^2, \quad (8)$$

where the sum is only over the movies that have been rated $i : r(i, j) = 1$ and for recommender systems normalizing by the number of movies is not needed (crossed variable above).

This is then extended to find the $w^{(n_u)}, b^{(n_u)}$ parameters for each user using Eq. (8):

$$J\left(w^{(1)}, \dots, w^{(n_u)}; b^{(j)}, \dots, b^{(n_u)}\right) = \sum_{j=1}^{n_u} J(w^{(j)}, b^{(j)}) \quad (9)$$

2.1.2 Algorithm

What if we do not have movie features?

How can we construct these features in our model?

Definition

Collaborative filtering is a powerful recommender system technique that predicts user preferences by finding similarities between users or items, predicting user recommendations based on behavior of similar users, without item features

Example

Now lets rewrite the previous data but without knowing what the movie features are:

Movie	Alice (1)	Bob (2)	Carol (3)	Dave (4)	x_1 (romance)	x_2 (action)
Love at last	5	5	0	0	?	?
Romance forever	5	?	?	0	?	?
Cute puppies of love	?	4	0	?	?	?
Nonstop car chases	0	0	5	4	?	?
Swords vs. karate	0	0	5	?	?	?

where we have already found some parameters for each user in our recommender system.

- Given some parameter set for each user, we can make a reasonable estimate of what the feature sets are for each movie that would keep the ratings the same
- This is possible because we have multiple ratings(values) from all users

Formalism

Given the above example we can construct the cost function to find the features given user ratings:

Given $w^{(1)}, b^{(1)}, \dots, w^{(n_u)}, b^{(n_u)}$

to learn $x^{(i)}$:

$$J(x^{(i)}) = \frac{1}{2} \sum_{j:r(i,j)=1} \left(w^{(j)} \cdot x^{(i)} + b^{(j)} - y^{(i,j)} \right)^2 + \frac{\lambda}{2} \sum_{k=1}^n \left(x_k^{(j)} \right)^2, \quad (10)$$

And again to learn all the features we can sum over all movies(items) using Eq. (10):

$$J(x^{(1)}, \dots, x^{(n_m)}) = \sum_{i=1}^{n_m} J(x^{(i)}) \quad (11)$$

Note

Now we can put the formalism from Eqs. (9) and (11) together to form the **collaborative filtering** cost function (notice the first sum in both of these equations are just over the movies that have been rated and are the same):

$$J(w, b, x) = \frac{1}{2} \sum_{(i,j):r(i,j)=1} \left(w^{(j)} \cdot x^{(i)} + b^{(j)} - y^{(i,j)} \right)^2 + \frac{\lambda}{2} \sum_{j=1}^{n_u} \sum_{k=1}^n \left(w_k^{(j)} \right)^2 + \frac{\lambda}{2} \sum_{j=1}^{n_m} \sum_{k=1}^n \left(x_k^{(j)} \right)^2, \quad (12)$$

to be minimized to find the parameters w, b and features x simultaneously by minimizing Eq. (12) using gradient descent while updating x as well.

2.1.3 Generalization to binary labels**Definition**

Binary labels for recommender systems could take the form of like/dislike, if the item was clicked on/not, favorited items, etc

- This is analogous to liking movies on a streaming service

To generalize the collaborative filtering approach to binary classifications we follow the same procedures learned in course 1 where we went from regression model to logistic models.

- Previously we used linear regression: Predict $\vec{y}^{(i,j)}$ as $\vec{w}^{(j)} \cdot \vec{x}^{(i)} + \vec{b}^{(j)}$
- For binary labels: Predict probability $\vec{y}^{(i,j)} = 1$ as $g(\vec{w}^{(j)} \cdot \vec{x}^{(i)} + \vec{b}^{(j)})$ where $g(z)$ is the sigmoid function.

Formalism

Remember that the cost function for binary labels is the Loss function (binary cross entropy) instead of the squared errors function (see course 1 notes for details):

$$J(w, b, x) = \sum_{(i,j):r(i,j)=1} L \left(f_{(w,b,x)}(x), y^{(i,j)} \right) \quad (13)$$

where

$$f_{(w,b,x)}(x) = g \left(w^{(j)} \cdot x^{(i)} + b^{(j)} \right)$$

mentioned above.

2.2 Recommender System Implementation

Note

Mean normalization(discussed in course 1) of the parameters becomes essential in recommender systems with the cost function described by Eq. (12) and runs more efficiently

- For new users(no ratings yet) the parameters are initialized to be zero because the first summation is only over movies rated and regularization will push the parameter to zero.
- This is an issue because new users will have recommendations of zero for all items
- This replaces the zero initialization with the average rating among users for new users with no/little history

Formalism

How do we perform the mean normalization for recommender systems?

- We can define a $n_m \times n_u$ matrix $\mathbf{R}$
- Calculate the mean/average rating per movie stored in a vector $\boldsymbol{\mu}$
- Retrieve the mean normalized matrix $\mathbf{M}$ where each element is subtracted by the mean value ($R_{ij} - \mu_i$)

So then for user j , on movie i predict:

$$w^{(j)} \cdot x^{(i)} + b^{(j)} + \mu_i$$

so we get non-zero predictions for new users.

2.2.1 Tensorflow implementation

Definition

Auto Diff is an algorithm within Tensorflow and other ML libraries that allows for automatic evaluation of derivatives

- This is useful to compute the derivative of the cost function for optimization algorithms like gradient descent and Adam
- We need to use this method because the conventional **Dense** layers do not apply to the recommender system to implement cost function manually

Example

To implement this directly in Tensorflow an example of the auto diff procedure is below with a simple cost function $J = (wx - 1)^2$ with gradient descent:

```
# parameters we want to optimize as tf variables
w = tf.Variable(3.0)
x = 1.0
y = 1.0 #target
alpha = 0.01

iterations = 30
for iter in range(iterations):
    # Tensorflows Gradient tape records steps used
    # to compute J, to enable auto diff
    with tf.GradientTape() as tape:
        fw = w*x #f(x)
        costJ = (fw - y)**2

    # Use the gradient descent tape to calculate gradients
    # with respect to w
    [dJdw] = tape.gradient( costJ, [w])

    # Run the step of gradient descent by updating w
    w.assign_add(-alpha * dJdw)
```


2.2.2 Finding related item

Formalism

How do we find related items for recommender systems to recommend related items?

The feature $x^{(i)}$ are quite hard to interpret.

To find other items related to it, find item k with $x^{(k)}$ similar to $x^{(i)}$

- To do this we can use a familiar formalism from clustering, namely the distance formalism
- We can compare the user item features to each other and find their nearest neighbors (items with most similar feature values) using our distance equation

$$\sum_{l=1}^n \left(x_l^{(k)} - x_l^{(i)} \right)^2 \quad (14)$$

2.2.3 Limitations of collaborative filtering

Note

The limitations

- **Cold start problem** is what we discussed previously on handling users with little or no history or items with little/no ratings
- **Using side information about items or users**, e.g., item genre, movie stars, etc and user demographics and preferences are highly valuable information that are not used in this algorithm

2.2.4 Assignment

Graded Assignment: Tensorflow implementation of movie recommender system using custom ML loops with collaborative filtering algorithm.

2.3 Content-based Filtering

Definition

Content-based filtering algorithm recommend items to user based on features of user and item to find a good match

- As opposed to collaborative filtering where recommendations are based on ratings of users who gave similar ratings as you
- Advantage now that we can use features of user, e.g., age, gender, country, etc, and features of items, e.g., movie year, genre, average ratings, etc

Formalism

To predict content-based recommendation for user j on item i some user features $x_u^{(j)}$ and item features $x_m^{(i)}$ (in this case item is movie so subscript m):

$$\mathbf{v}_u^{(j)} \cdot \mathbf{v}_m^{(i)}, \quad (15)$$

where

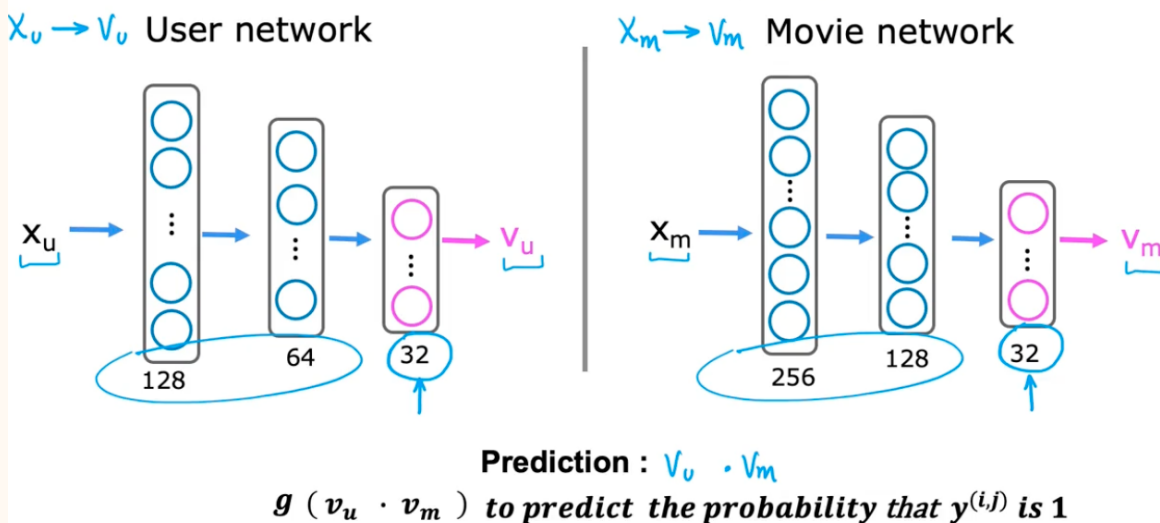
- $\mathbf{v}_u^{(j)}$ is a vector computed from user features $x_u^{(j)}$
- $\mathbf{v}_m^{(i)}$ is a vector computed from item features $x_m^{(i)}$
- $\mathbf{v}_u^{(j)}$ and $\mathbf{v}_m^{(i)}$ need to be the same size (dot product)
- we have dropped the constant term $b^{(j)}$ in the linear regression above.

2.3.1 Deep Learning Architecture

Example

The industry-ready solutions for content-based filtering are to use deep learning. Below shows an example of the architecture where:

- use two neural networks, one for user features and one for item features
- each network outputs the feature vector and item vector (same size output layer)
- Conduct the analysis step above Eq. (15) with neural network outputs



Formalism

How do we find the many parameters for the above neural networks?

We construct the cost function similar to Eq. (12)

$$J = \sum_{(i,j):r(i,j)=1} \left(v_u^{(j)} \cdot v_m^{(i)} - y^{(i,j)} \right)^2 + \text{NN regularization terms} \quad (16)$$

where we train the neural networks to have parameters that reduce the squared error of the output vectors.

Then how do we find movies similar to movie i

Again, to find similar vectors we apply the 'distance' formalism introduced in collaborative filtering Eq. (14) but with the item vectors

2.4 Advanced Implementation

How do we efficiently find recommendations for large sets of items?

For example,

- streaming platform could have thousands of movies
- there could be 1M+ ads to choose from
- commerce sites can have 10M+ items for sale

It becomes unfeasible to run a large neural network for every user everytime their needs to be a recommendation.

Definition

Two step handling called **Retrieval** and **Ranking**

For the **Retrieval**

- Generate large lists of plausible item candidates (could include movies the user may not like), e.g.,
 1. 10 similar movies to the last 10 movies a user watched using precomputed distance calculation (not heavy compute)
 2. top 10 movies for top 3 most viewed genres
 3. Top 20 movies in the country
- combine retrieval items into lists, remove duplicates and already watched/purchased items

For the **Ranking** step fine tune user preference of retrieved items

- Take retrieved list (significant reduction in size) and rank using learned model (deep learning architecture example)
- Display ranked items to user
- If you precompiled the v_m then inference only needs to be done on the user neural network and computationally light to make predictions

Recommendations for Retrieval step:

- Retrieving more items results in better performance but slower recommendations
- Analyze/optimize the trade-off through offline experimentation for relevant recommendations

2.4.1 Ethical issues

Choosing the goal of recommender systems

1. Movies most likely to be rated 5 stars
2. Products most likely to be purchased
3. Ads most likely to be clicked + high bid(advertiser)
4. Generate largest profit
5. video leading to maximum watch time

Looking at the above list 1 and 2 seem unobtrusive but 3, 4, and 5 may have some negative user repercussions., let's take a look at number 3 (ads)

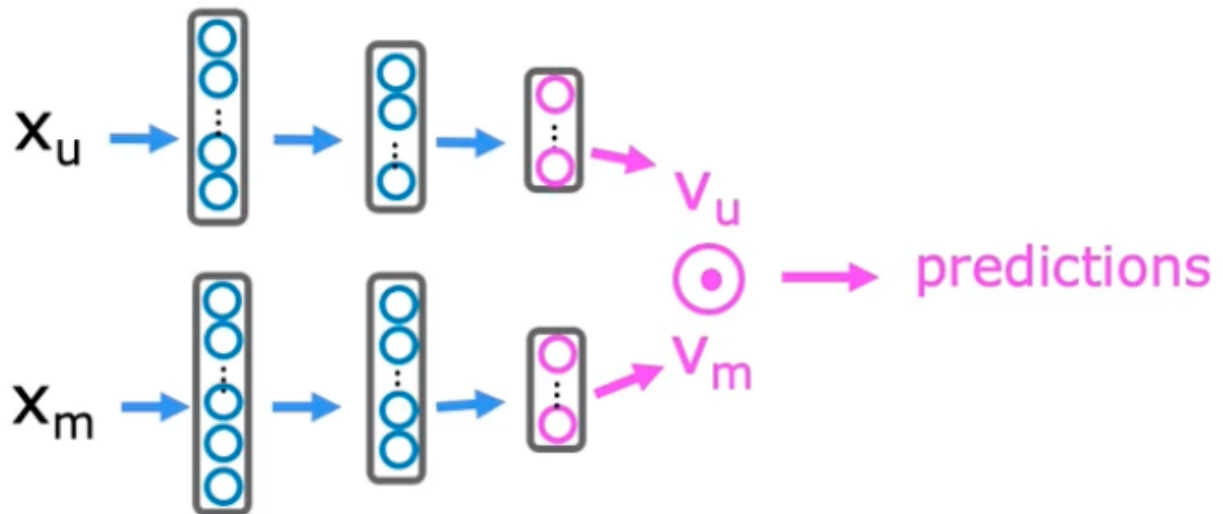
- **Good example:** Travel agency gives users good travel experiences → become more profitable → can make high bids, so is recommended more
- **Bad example:** payday loans are known to exploit underrepresented groups with high loan rates, squeeze customers more → more profit → higher bids/ more traffic
- **Possible solution:** do not accept ads from exploitative businesses (hard to gauge and quantify)

Other examples that may be brought up

- **Bad Example:** Maximizing user engagement has led to social media amplifying conspiracy theories and hate/toxicity
- **Possible Solution:** Filter out hate speech, fraud, scams or violent content(tricky to define)
- **Bad Example:** Maximizing business profits may not benefit user and not done transparently
- **Possible Solution:** Be transparent with users

2.4.2 Tensorflow implementation

For the implementation in tensorflow we can follow a few steps with code guided by the below image.



1. We will build the two neural networks as usual in tensorflow:

```
user_NN = tf.keras.models.Sequential([
    tf.keras.layers.Dense(256, activation='relu', name='name_of_layer1'),
    tf.keras.layers.Dense(128, activation='relu', name='name_of_layer2'),
    tf.keras.layers.Dense(32, activation='relu', name='name_of_layer3')
])
```

```
item_NN = tf.keras.models.Sequential([
    tf.keras.layers.Dense(256, activation='relu', name='name_of_layer1'),
    tf.keras.layers.Dense(128, activation='relu', name='name_of_layer2'),
    tf.keras.layers.Dense(32, activation='relu', name='name_of_layer3')
])
```

2. Create the user/item input and output to the base network

```
input_user = tf.keras.layers.Input(shape=num_user_features)
vu = user_NN(input_user)
vu = tf.linalg.l2_normalize(vu, axis=1) # improves efficiency

input_item = tf.keras.layers.Input(shape=num_item_features)
vm = user_NN(input_item)
vm = tf.linalg.l2_normalize(vum axis=1) # improves efficiency

# measure similarity of the two vector outputs
output = tf.keras.layers.Dot(axes=1)([vu,vm])
```

3. Specify the input and output of the model and cost function for minimization

```
model = tf.keras.Model([input_user, input_item], output)
cost_function = tf.keras.losses.MeanSquaredErrors()
```

2.5 Principal Component Analysis (optional)

Definition

Principal Component Analysis is an unsupervised learning algorithm to reduce the number of features to be able to visually represent the data

- Typically used by data scientists to reduce large data sets to 2 or 3 principal components for visualization
- can be used for compression (outdated)
- can be used in older ML architectures like svm (outdated)

2.6 Implementation

Course Labs

- Principal Component Analysis
 - Implement PCA in code to reduce high dimensional data set and visualize the chosen 2D subspace for explorative data analysis

Graded Programming Assignments

- Collaborative Filtering Recommender System
 - Implement collaborative filtering to build a recommender system for movies
 - Use tensorflow autodiff and custom learning loop to implement gradient descent
- Deep Learning for Content-based Filtering
 - Implement content-based filtering using a neural network to build a recommender system for movies
 - Implement a deep learning network with two neural networks to predict user movie preferences

3. Reinforcement Learning

Definition

Reinforcement learning is a learning algorithm that takes a state s , set of inputs of current state from environment, and aims to fulfill an action a , and receiving rewards or punishments x (feedback) to maximize cumulative reward over-time like trial-and-error learning.

Example use cases:

- Controlling robots
- Factory optimization (maximize efficiency)
- Financial trading (stock execution)
- Playing games and video games

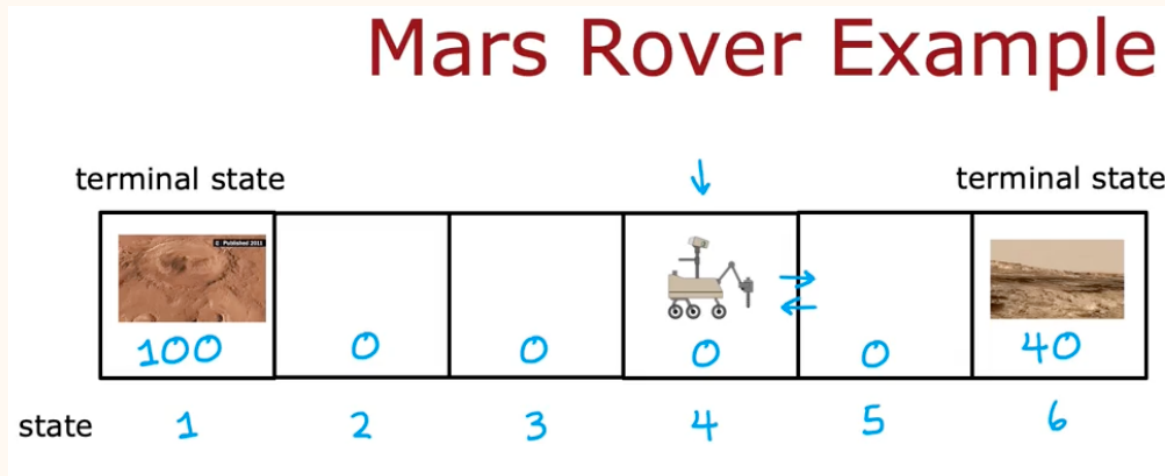
3.1 Formalism

Example

Simplified Mars Rover example:

The image below shows a simplified diagram to show qualitatively the formalism for reinforced learning

- There are 6 states s for the robot to determine whether to go left or right to meet some actions(research the most interesting terrains) based on rewards(number in each state, terrains that are most interesting to sample)
- The state 1 and 6 are terminal states so when reached the day ends or the goal is met
- The robot has to take actions and get rewarded or penalized for these actions, where each step is defined by the initial state, action, reward, and final state $(s, a, R(s), s')$
- Example path going left from state 4, $4 \rightarrow 3 \rightarrow 2 \rightarrow 1$, will accumulate 100 reward points



Definition

The **Return** of the reinforcement learning tells the return for actions over time, e.g., if the return is large but take more time it may be more beneficial to take the action with smaller return over smaller time period.

- The return can then be defined as a sum of actions times a **discount factor** in a series representation written succinctly:

$$Return = \sum_{i=1}^{n_a} \gamma^{i-1} R_i, \quad (17)$$

where n_a is the number of actions taken, $\gamma < 1$ and is the discount factor, and R_i is the i -th reward.

- We can also see that γ increasingly reduces each future return (increasing powers) which has the effect of penalizing longer sequences

The **Policy** Π in reinforcement learning maps the state to the action that should be taken to maximize return, e.g., in the mars rover example if the policy were 'in state 2 go left' translates $\Pi(2) = \leftarrow$

- Generally the mapping to the action is

$$\Pi(s) = a$$

Formalism

The above elements of the reinforcement learning can be summarized as a **Markov Decision Process (MDP)** where the future only depends on the current state

[Agent, Π] $\xrightarrow{\text{takes an action, } a}$ [Environment after action] $\xrightarrow{\text{evaluate reward, } R \text{ from initial state, } s}$ Repeat

3.2 State action value function

Definition

State action value function denoted by $Q(s, a)$ outputs the return Eq. (17) of an initial state $s_{initial}$ and initial action $a_{initial}$ for the optimal path of actions after $a_{initial}$

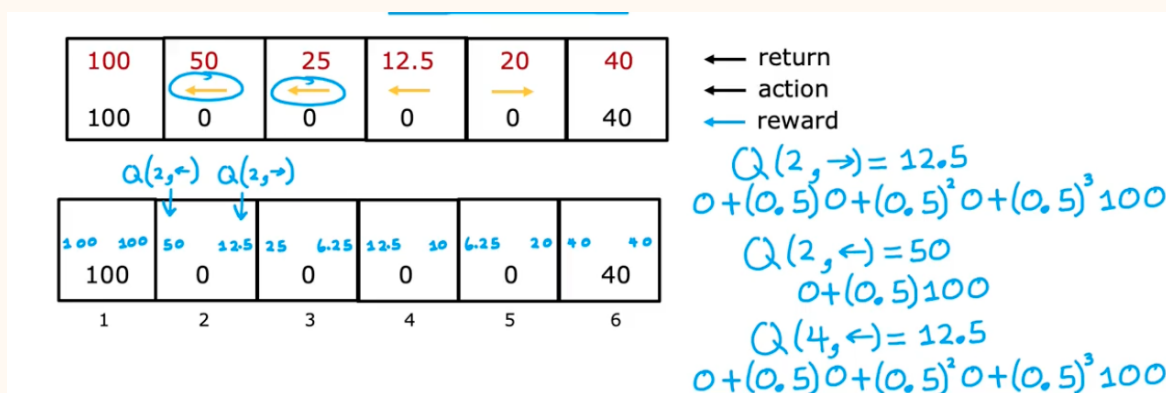
- The logic may seem circular because how do we know the optimal path of actions after some initial state and action?

My Guess: Probably minimize/maximizes this quantity for learning algorithm.

- **Note:** Sometimes called **Optimal Q function** and denoted Q^* in literature

Example

Looking at the mars rover example again we can calculate $Q(s, a)$ of state s if you take action a and behave optimally afterwards, where the image below computes this function with discount factor $\gamma = 0.5$ for the 'left' or 'right' initial action at each state



For example, for $Q(2, \rightarrow)$ the optimal path is to reach state 1 and so is $2 \rightarrow 3 \leftarrow 2 \leftarrow 1$ and calculating the return using Eq. (17) gives

$$Q(2, \rightarrow) = 0 + 0.5 * 0 + (0.5)^2 * 0 + (0.5)^3 * 100 = 12.5$$

Formalism

It turns out that:

- The best possible return from state s is $\max_a Q(s, a)$
- The best possible action in the state s is the action a that gives $\max_a Q(s, a)$

So the optimal policy for all states can be written as:

$$\Pi(s) = a \quad \text{for action that gives } \max_a Q(s, a) \quad (18)$$

Example of above For picking the actions we can see by the image above of the rover problem that depending on a state the best action to take maximizes the initial return,

- In state 4 $Q(4, \leftarrow) = 12.5$, $Q(4, \rightarrow) = 10$, choosing the maximum return for action gives maximum return for action path

Optional Lab: Change rewards function, discount value to calculate $Q(s, a)$ for all states in our mars rover example.

3.2.1 Bellman Equation

Formalism

The **Bellman equation** is an equation that calculates the state action value function $Q(s, a)$ given the below set of variables:

Variable	Definition
s	current state
a	current action
$R(s)$	rewards of current action
s'	state after taking action a
a'	action taken in state s

it is written as:

$$Q(s, a) = R(s) + \gamma \max_{a'} Q(s', a') \quad (19)$$

where for a terminal state $Q(s, a) = R(s)$ because the second term doesn't exist.

Meaning of Bellman equation

- $R(s)$ term tells us the immediate reward
- $\gamma \max_{a'} Q(s', a')$ tells us the return from behaving optimally starting from state s'

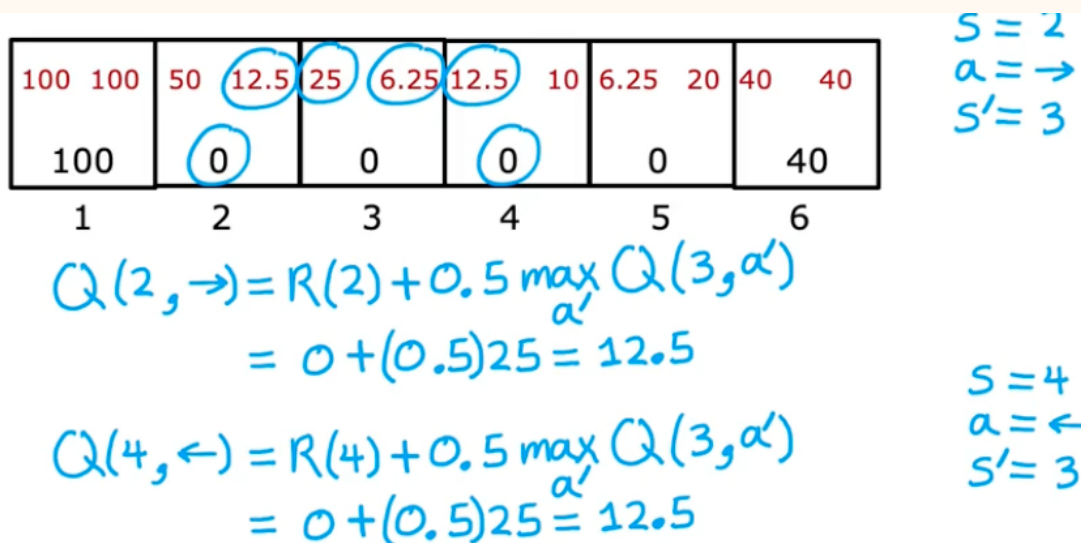
Knowing this and comparing to our returns equation ?? we can rewrite the Bellman equation using the returns definition

$$\begin{aligned} Q(s, a) &= \sum_{i=1}^{n_a} \gamma^{i-1} R_i(s) \\ &= R(s) + \gamma \sum_{i=2}^{n_a} \gamma^{i-2} R(s'_i) \end{aligned} \quad (20)$$

where for optimal path $\sum_{i=2}^{n_a} \gamma^{i-2} R(s'_i) = \max_{a'} Q(s', a')$

Example

Again we can calculate some values for the Bellman equation to verify our understanding of how it works. The below image shows the mars rover examples and some calculated values using Eq. (19).

**3.2.2 (Optional) Stochastic environment**

In stochastic (random) processes there is a probability of deviating from the expected path. The example of the land rover can be extended to understand this

- In the land rover example if the terrain makes it difficult to perform the expected outcome we can formulate this into a stochastic process
- For a given state if the robot wants to go left there may be a small probability (10%) that it may go right instead due to environment (e.g., mudslide)
- Now the returns will take many potential paths depending on the random process

To account for a random process with multiple probable paths we need to maximize the return for all potential paths by taking the average or expected value of all possible returns:

$$\text{Expected Return} = E \left[\sum_{i=1}^{n_a} \gamma^{i-1} R_i \right] \quad (21)$$

where $E[\cdot]$ is the expected value or average value of all possible paths.

So the Bellman equation can now be modified with s' as a random variable

$$Q(s, a) = R(s) + \gamma E \left[\max_{a'} Q(s', a') \right] \quad (22)$$

3.3 Continuous State Space

Definition

A **continuous state space** refers to a state function with continuous variables, e.g., for vehicle automation the state space would be the positions and velocities $s = (x, y, \theta, \dot{x}, \dot{y}, \dot{\theta})$ that take continuous values

Example

Lunar lander example with actions:

- do nothing
- left thruster
- main thruster
- right thruster

with the goal of landing between two designated flags. (We will see the implementation of this in a practice lab.)

The state space can be described by the vector

$$s = (x, y, \theta, \dot{x}, \dot{y}, \dot{\theta}, l, r)$$

describing the position and orientations, their rate of change (velocity), and (l, r) denoting whether the left or right legs have landed on the ground.

The **reward function** is then constructed as:

- Getting to landing pad: 100-140
(additional rewards for moving towards/away from pad)
- Crash: -100
- Soft landing: +100
- Leg grounded: +10
- Fire main engine: -0.3
- Fire side thruster: -0.03

where the last two small penalties allow for more efficient fuel use while landing.

The problem statement is:

Learn a policy π that, given s (defined above) picks an action $a = \pi(s)$ so as to maximize the return with discount factor $\gamma = 0.985$

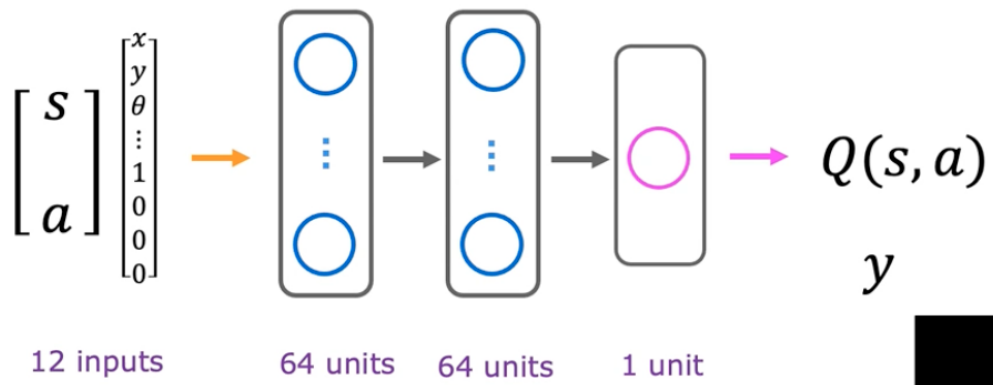
How can we solve the above problem statement?

- Implementation using deep reinforcement learning:

- In state s , use neural network to compute

$$Q(s, \text{nothing}), Q(s, \text{left}), Q(s, \text{main}), Q(s, \text{right})$$

Pick the action that maximizes $Q(s, a)$



- Creating the data set for neural network implementation

- Use bellman equation Eq. (19) to create a data set with input vector $\vec{x} = (s, a)$ and the output y from the computed value
 - We initially take a random guess for the $Q(s', a')$ in Eq. (19) and see later that this converges to optimal function with further iterations.

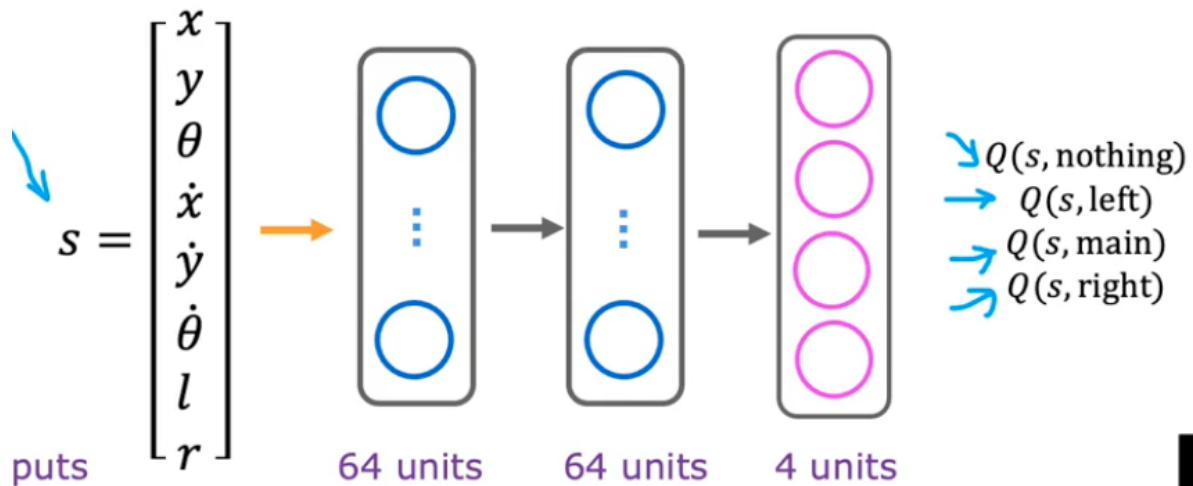
Formalism

Summarize into the Learning Algorithm (Deep Q Network (DQN)):

- Initialize neural network randomly with an initial guess for $Q(s, a)$
- Repeat {
 - Take actions in the lunar lander. Get tuple $(s, a, R(s), s')$
 - Store 10,000 most recent tuples (Replay Buffer) for efficiency
 - Train neural network:
 - Create training set of 10,000 examples using $\vec{x} = (s, a), y = R(s) + \gamma \max_a Q(s', a')$
 - Train Q_{new} such that $Q_{new}(s, a) \approx y$
 - Set $Q = Q_{new}$

3.4 Algorithm Improvements

- The **neural network architecture** can be optimized to output all state functions by changing the final layer



How do we choose actions while still learning?

Definition

- ϵ Greedy Policy

In the algorithm formalism above when we take actions we randomly initialize $Q(s, a)$ instead we could:

- Pick the action a that maximizes $Q(s, a)$ "Greedy/Exploitation"
- **Best Choice:** With probability 0.95, the action a that maximizes $Q(s, a)$, with probability 0.05, pick an action randomly "Exploration"

It turns out choosing to maximize the action for $Q(s, a)$ may cause some bias in the model and the best option by randomly choosing the action sometimes allows for the network to overcome this potential bias.

The best choice is called a ϵ **greedy policy**.

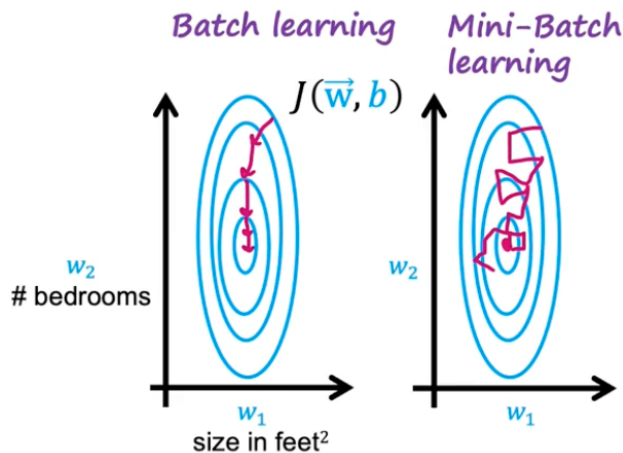
Other Methods to increase efficiency and convergence

Definition

- **Mini-batch** learning uses a subset of the total example set for each iteration of gradient descent

Advantages of mini-batch in supervised learning and our reinforcement learning algorithm:

- This is an effective algorithm to speed up gradient descent of large data sets (remember we have to sum over all examples for each gradient descent)
- Because its a subset of examples getting to the minimum becomes much more noisy but averaging over data in batches it will find the minimum



- On large datasets this turns out to be more efficient even though we may take more steps to reach the best parameters, because each step is significantly lighter than using all examples
- In reinforcement learning algorithm above instead of creating 10,000 training examples we may choose 1,000 for a mini-batch training set formation while learning

Definition

The **soft update** method can be used in the update step of the reinforcement learning algorithm when we set $Q = Q_{new}$ to assure that the Q_{new} is in fact better than the previous iteration.

To do this when updating Q we only take a small change to the original Q by updating the linear parameters of the training data by a small amount

$$W = 0.01W_{new} + 0.99W, B = 0.01B_{new} + 0.99B$$

This makes the reinforcement learning algorithm converge more reliably.

3.5 Implementation

Course Labs

- State Function Value Function Example
 - Modify the mars rover example to see how the values of $Q(s, a)$ will change depending on the rewards and discount factor changing
 - Also provides functionality to include a stochastic process by introducing a misstep

probability.

Graded Programming Assignments

- Deep Q-Learning: Lunar Lander Simulation
 - Train an agent to land a lunar lander safely on a landing pad on the surface of the moon provided the outline and instruction discussed in the notes above.
 - Code is depreciated and needs to be updated, I completed this assignment internally though Coursera.

Final assignment making a reinforcement learning model to solve the lunar mars landing problem described above.